



Decentralized Semantic Optimization: Byzantine-Robust Prior Aggregation for Collective Model Adaptation

Zach Kelling

Hanzo AI

December 2023

Abstract

We present **Decentralized Semantic Optimization (DSO)**, a protocol for sharing and aggregating experiential priors across distributed language model agents without parameter updates. While Active Semantic Optimization (ASO) enables individual agents to adapt via training-free Product-of-Experts decoding, DSO extends this to the multi-agent setting by defining a Byzantine-fault-tolerant aggregation protocol over compressed semantic priors.

DSO introduces three key mechanisms: (i) a *three-layer semantic storage architecture* combining on-chain registry, content-addressed off-chain storage (IPFS/Arweave), and local replica databases; (ii) a *Byzantine-robust weighted median aggregation* scheme with formal guarantees under $f < n/3$ Byzantine faults; and (iii) a *Sybil-resistant gossip protocol* with stake-weighted propagation and quality-based filtering.

We prove that DSO aggregation converges to the honest-majority posterior at rate $O(1/\sqrt{n})$ where n is the number of honest participants, and that the protocol tolerates up to $f < n/3$ Byzantine nodes while maintaining ε -optimal prior quality. Empirical evaluation on multi-agent code generation demonstrates **15.2% improvement** when 10 agents share priors vs. isolated operation, with resilience to 20% Byzantine participants (degradation $< 0.9\%$).

1 Introduction

The Active Semantic Optimization (ASO) framework [9] demonstrates that frozen language models can be adapted to new tasks via compressed experiential priors applied at decode time. However, ASO operates in isolation: each agent builds its own prior bank from its own rollouts, unable to benefit from the experiences of other agents working on similar tasks.

Traditional federated learning [16] addresses multi-agent learning by aggregating gradients or model parameters, but this approach is fundamentally incompatible with training-free adaptation. Parameter-efficient methods like federated LoRA [23] reduce communication costs but still require gradient computation on each participant.

The Sharing Problem. Consider n agents independently running ASO on related tasks. Each produces compressed priors encoding what works and what does not for their specific sub-problems. These priors contain transferable knowledge: an agent that discovers “always handle `None` inputs before processing” benefits all agents working on similar Python code. Yet without a sharing mechanism, this knowledge remains siloed.

The Trust Problem. In a decentralized setting, some participants may be Byzantine: submitting random noise, targeted poisoning attacks, or Sybil identities that vote for malicious priors. Any aggregation protocol must be robust to such adversarial behavior while still extracting value from honest contributions.

Our Contribution. DSO solves both problems simultaneously via:

1. A three-layer storage architecture separating integrity (on-chain), availability (IPFS/Arweave), and performance (local replicas).
2. Byzantine-robust weighted median aggregation with formal fault tolerance guarantees.
3. Sybil-resistant gossip with stake-weighted propagation and quality-gated filtering.
4. Integration with Proof-of-AI (PoAI) consensus for attestation-based quality verification and incentive alignment.

Paper Outline. Section 2 provides background on ASO and Byzantine consensus. Section 3 describes the three-layer storage architecture. Section 4 specifies the DSO protocol. Section 5 formalizes Byzantine-robust aggregation. Section 6 analyzes Sybil resistance. Section 7 specifies the gossip protocol. Section 8 proves convergence results. Section 9 describes incentive mechanisms. Section 10 presents experimental evaluation. Section 11 discusses related work. Section 12 concludes.

2 Background

2.1 ASO Experiential Priors (Recap)

In ASO [9], each experiential prior E_m consists of:

- A 1-bit quantized expert factor $\hat{\Delta}_m \in \{-1, +1\}^{|\mathcal{V}_m|}$ with scale $\alpha_m > 0$.
- A quality score $q_m \in (0, 1)$ estimating pattern reliability.
- A context matching predicate $C_m(x, y_{<t}) \in \{0, 1\}$ specifying when the prior applies.
- Metadata: task embedding $e_m \in \mathbb{R}^d$, timestamp, source attribution.

At decode time, priors contribute to the Product-of-Experts distribution:

$$\pi_{\text{ASO}}(y_t \mid x, y_{<t}) \propto \pi_{\theta}(y_t \mid x, y_{<t}) \prod_{m=1}^M \phi_m(y_t \mid x, y_{<t})^{\eta_m}, \quad (1)$$

where $\eta_m = \log(q_m/(1 - q_m))$ and ϕ_m is constructed from $\hat{\Delta}_m$.

2.2 Byzantine Fault Tolerance

A distributed protocol is *f-Byzantine-fault-tolerant* if it maintains correctness properties when up to f of n participants behave arbitrarily [13]. The classical bound for agreement-based protocols is $f < n/3$ [19].

Definition 1 (Byzantine Node). *A node i is Byzantine if it may deviate from the protocol specification in any way, including:*

1. *Submitting arbitrary (possibly adversarial) prior data.*
2. *Withholding or selectively propagating messages.*
3. *Colluding with other Byzantine nodes.*
4. *Creating multiple identities (Sybil attack).*

2.3 Weighted Median

The weighted median is a robust estimator that tolerates up to half the total weight in outliers [18].

Definition 2 (Weighted Median). *Given values $\{x_i\}_{i=1}^n$ with non-negative weights $\{w_i\}_{i=1}^n$ summing to $W = \sum_i w_i$, the weighted median is:*

$$\text{wmed}(\{x_i, w_i\}) = \inf \left\{ x : \sum_{i: x_i \leq x} w_i \geq W/2 \right\}. \quad (2)$$

The weighted median has a breakdown point of $1/2$ in total weight, meaning an adversary must control more than half the total weight to move the estimate arbitrarily.

3 Three-Layer Semantic Storage Architecture

DSO employs a three-layer architecture optimizing for different properties at each level.

3.1 Layer 1: On-Chain Registry (Integrity)

The ExperienceRegistry smart contract stores metadata and integrity proofs for each submitted prior. No prior data is stored on-chain; only cryptographic commitments.

Definition 3 (Registry Entry). *A registry entry $\mathcal{R}_i$ consists of:*

$$\mathcal{R}_i = (\text{merkleRoot}_i, \text{cid}_i, \text{schema}_i, q_i, \text{addr}_i, D_i, t_i), \quad (3)$$

where merkleRoot_i is the Merkle root of the prior data, cid_i is the content identifier on IPFS/Arweave, schema_i identifies the decomposition schema, q_i is the quality score, addr_i is the submitter's address, D_i is the bond amount, and t_i is the submission timestamp.

Contract Interface.

```
interface IExperienceRegistry {
    struct Entry {
        bytes32 merkleRoot;
        string  cid;           // IPFS or Arweave content ID
        uint64  schema;        // Schema version identifier
        uint64  quality;        // Quantized quality [0, 1000]
        address submitter;      // Submitter address
        uint256 bond;           // Staked bond amount
        uint64  timestamp;      // Submission block timestamp
    }

    function submit(Entry calldata e)
        external payable returns (uint256 id);

    function voteQuality(uint256 id, uint64 score)
        external;

    function challenge(uint256 id, bytes calldata proof)
        external returns (bool slashed);

    function aggregate(uint256[] calldata ids)
        external returns (bytes32 aggregateMerkleRoot);

    function getEntry(uint256 id)
        external view returns (Entry memory);

    function getEntriesBySchema(uint64 schema)
```

```

    external view returns (uint256[] memory ids);
}

```

3.2 Layer 2: Content-Addressed Storage (Availability)

Prior data is stored on IPFS [2] or Arweave [21] for persistent availability. Each prior is serialized as a Protocol Buffer message:

```

message ExperiencePrior {
    bytes    delta_signs = 1;      // Packed 1-bit signs
    float    scale = 2;           // Per-expert scale factor
    float    quality = 3;         // Quality score
    bytes    context_predicate = 4; // Serialized matching fn
    bytes    task_embedding = 5;  // d-dimensional embedding
    uint64    vocab_size = 6;     // |V_m|
    repeated uint32 vocab_indices = 7; // Sparse support
    bytes    merkle_proof = 8;    // Proof of inclusion
}

```

The content identifier (CID) is the SHA-256 hash of the serialized message, ensuring deduplication and integrity verification.

3.3 Layer 3: Local Replica Database (Performance)

Each node maintains a local LanceDB [14] instance containing replicas of high-quality priors. The local database supports:

- **Vector search:** Nearest-neighbor queries over task embeddings for prior retrieval in $O(\log N)$ time.
- **CRDT semantics:** Conflict-free merge of priors from multiple sources using last-writer-wins with quality-based tie-breaking.
- **Garbage collection:** Automatic eviction of priors with quality below $q_{\min}$ or age exceeding $T_{\max}$.
- **Offline operation:** Full functionality without network access using cached priors.

Definition 4 (Replica Consistency). *A local replica L_i is τ -consistent with the registry if for every entry $\mathcal{R}_j$ with $q_j \geq q_{\text{fetch}}$ and $t_j \leq t_{\text{now}} - \tau$, the corresponding prior data is present in L_i and its Merkle proof verifies against merkleRoot_j .*

3.4 Cross-Layer Integrity

The three layers are connected by cryptographic commitments:

Proposition 1 (End-to-End Integrity). *For any prior with registry entry $\mathcal{R}_i$, a verifier can confirm that the local replica data E_i is authentic by:*

1. *Computing the Merkle root of E_i and comparing with merkleRoot_i from the on-chain registry.*
2. *Verifying the CID of the serialized data matches cid_i .*

3. *Checking the Merkle proof within the serialized data.*

Any modification to the prior data is detected with probability $1 - 2^{-256}$ (collision resistance of SHA-256).

4 DSO Protocol Specification

4.1 Roles

The DSO protocol defines five roles:

1. **Agents:** Run local ASO, extract experiential priors, submit to registry. Each agent has a keypair (pk_i, sk_i) and stake S_i .
2. **Aggregators:** Compute Byzantine-robust aggregations of submitted priors. May be a subset of agents or dedicated nodes.
3. **Validators:** Verify PoAI attestations, vote on quality scores, execute slashing for malicious submissions.
4. **Storage Providers:** Pin prior data on IPFS/Arweave, earn propagation rebates.
5. **Registry:** Smart contract managing entries, bonds, quality votes, and aggregation requests.

4.2 Protocol Phases

Algorithm 1 DSO Protocol: Full Lifecycle

Require: Agent set $\mathcal{A} = \{a_1, \dots, a_n\}$, task distribution $\mathcal{D}$

Ensure: Aggregated prior bank $\mathcal{E}_{\text{agg}}$

```

1: // Phase 1: Local Prior Generation
2: for each agent  $a_i \in \mathcal{A}$  in parallel do
3:   Sample task  $x \sim \mathcal{D}$ 
4:    $\mathcal{E}_i \leftarrow \text{ASO}(x, \pi_\theta)$  ▷ Local TF-GRPO + PoE
5: end for
6: // Phase 2: Submission
7: for each agent  $a_i$  with priors  $\mathcal{E}_i$  do
8:   for each prior  $E \in \mathcal{E}_i$  do
9:     cid  $\leftarrow \text{IPFS.Pin}(\text{Serialize}(E))$ 
10:    root  $\leftarrow \text{MerkleRoot}(E)$ 
11:    id  $\leftarrow \text{Registry.Submit}(\text{root}, \text{cid}, q_E, D)$  ▷ Bond  $D$  required
12:   end for
13: end for
14: // Phase 3: Quality Verification
15: for each validator  $v_j$  do
16:   Sample subset of submissions  $\mathcal{S}_j$ 
17:   for each id  $\in \mathcal{S}_j$  do
18:      $E \leftarrow \text{IPFS.Fetch}(\text{cid}_{\text{id}})$ 
19:     Verify Merkle proof
20:      $q' \leftarrow \text{EvaluateQuality}(E, \text{holdout tasks})$ 
21:     Registry.VoteQuality(id,  $q'$ )
22:   end for
23: end for
24: // Phase 4: Aggregation
25: ids  $\leftarrow \text{Registry.GetEntriesBySchema}(\text{schema})$ 
26: Filter: ids  $\leftarrow \{\text{id} : q_{\text{id}} \geq q_{\min}\}$ 
27:  $\mathcal{E}_{\text{agg}} \leftarrow \text{ByzantineAggregate}(\text{ids})$  ▷ Section 5
28: // Phase 5: Gossip Propagation
29: for each node  $n_k$  do
30:   Subscribe to registry events
31:   Fetch and verify high-quality priors
32:   Merge into local LanceDB ▷ Section 7
33: end for
34: return  $\mathcal{E}_{\text{agg}}$ 

```

4.3 Schema Decomposition

Aggregation requires a common decomposition schema $\mathcal{S}$ so that priors from different agents can be compared element-wise.

Definition 5 (Schema). *A schema $\mathcal{S} = \{s_1, \dots, s_K\}$ partitions the token-level expert factor space into K bins. Each bin s_k corresponds to either:*

1. A **token bin**: A subset of vocabulary tokens grouped by semantic similarity (e.g., all Python keywords, all arithmetic operators).
2. An **embedding centroid**: A region in the task embedding space identified by k -means clustering.
3. A **context type**: A category of generation contexts (e.g., function signature, loop body, error handling).

Each submitted prior E_i is decomposed under the schema:

$$E_i = \{\Delta_i^{(s)}\}_{s \in \mathcal{S}}, \quad (4)$$

where $\Delta_i^{(s)} \in \{-\alpha_i, 0, +\alpha_i\}^{|s|}$ is the 1-bit quantized factor restricted to schema element s .

5 Byzantine-Robust Aggregation

5.1 Threat Model

Assumption 1 (Threat Model). *Of n total participants, at most $f < n/3$ are Byzantine. Byzantine nodes may:*

1. *Submit arbitrary prior data (random noise, targeted attacks).*
2. *Collude to coordinate attacks.*
3. *Create Sybil identities (bounded by stake requirements).*
4. *Selectively withhold or delay messages.*

Honest nodes follow the protocol and have priors with quality $q_i > 0.5 + \delta$ for some $\delta > 0$.

5.2 Weighted Median Aggregation

For each schema element $s \in \mathcal{S}$ and each token position v within s , the aggregate is computed as:

$$\tilde{\Delta}^{(s)}(v) = \text{wmed} \left(\{\Delta_i^{(s)}(v)\}_{i=1}^n; \{w_i\}_{i=1}^n \right), \quad (5)$$

where the weight for participant i is:

$$w_i = S_i \cdot q_i \cdot \mathbb{I}[q_i \geq q_{\min}], \quad (6)$$

with S_i the stake and q_i the validated quality score.

Theorem 2 (Byzantine Robustness of Weighted Median). *Under Assumption 1, if the total honest stake exceeds twice the total Byzantine stake:*

$$W_H = \sum_{i \in \text{honest}} w_i > 2 \sum_{j \in \text{Byzantine}} w_j = 2W_B, \quad (7)$$

then the weighted median (Eq. 5) equals a value submitted by an honest node for every schema element and token position.

Proof. Consider any schema element s and token position v . Let $\{\Delta_i^{(s)}(v)\}_{i \in H}$ be the honest submissions and $\{\Delta_j^{(s)}(v)\}_{j \in B}$ be the Byzantine submissions, with corresponding weights $\{w_i\}_{i \in H}$ and $\{w_j\}_{j \in B}$.

The total weight is $W = W_H + W_B$. The weighted median selects the value x such that the total weight of values $\leq x$ is $\geq W/2$. For the median to equal a Byzantine value $\Delta_j^{(s)}(v)$ for $j \in B$, the Byzantine nodes would need to shift the weight balance so that strictly more than $W/2$ weight lies on one side. This requires:

$$W_B > W/2 - W_H/2 = (W_H + W_B)/2 - W_H/2 = W_B/2, \quad (8)$$

which is trivially true, but for the median to be *moved away from all honest values*, we need $W_B \geq W_H/2$. Since $W_H > 2W_B$ implies $W_B < W_H/2$, this is impossible. Therefore, the median must lie within the range of honest submissions.

Since honest submissions are drawn from $\{-\alpha_i, 0, +\alpha_i\}$ (1-bit quantized), the median equals one of the honest values. $\square$

5.3 Coordinate-Wise Aggregation

For multi-dimensional prior factors, we apply the weighted median coordinate-wise:

Algorithm 2 Byzantine-Robust Prior Aggregation

Require: Submissions $\{E_i\}_{i=1}^n$, weights $\{w_i\}_{i=1}^n$, schema $\mathcal{S}$

Ensure: Aggregated prior $\tilde{E}$

- 1: **for** each schema element $s \in \mathcal{S}$ **do**
 - 2: **for** each token position v in s **do**
 - 3: $\tilde{\Delta}^{(s)}(v) \leftarrow \text{wmed}(\{\Delta_i^{(s)}(v)\}_{i=1}^n; \{w_i\})$
 - 4: **end for**
 - 5: $\tilde{\alpha}^{(s)} \leftarrow \text{wmed}(\{\alpha_i^{(s)}\}_{i=1}^n; \{w_i\})$
 - 6: **end for**
 - 7: $\tilde{q} \leftarrow \text{wmed}(\{q_i\}_{i=1}^n; \{w_i\})$
 - 8: $\tilde{E} \leftarrow \text{Assemble}(\{\tilde{\Delta}^{(s)}, \tilde{\alpha}^{(s)}\}_{s \in \mathcal{S}}, \tilde{q})$
 - 9: **return** $\tilde{E}$
-

5.4 Aggregation Quality Guarantee

Theorem 3 (Quality Preservation). *Let $q^* = \text{med}(\{q_i\}_{i \in H})$ be the median quality of honest submissions. Under Assumption 1, the aggregated prior quality satisfies:*

$$\tilde{q} \geq q^* - O\left(\frac{1}{\sqrt{|H|}}\right), \quad (9)$$

where $|H| = n - f$ is the number of honest participants.

Proof. By Theorem 2, the weighted median quality $\tilde{q}$ lies within the range of honest quality scores. The median of $|H|$ honest scores concentrates around the population median at rate $O(1/\sqrt{|H|})$ by the central limit theorem applied to order statistics [6]. Since $\tilde{q}$ is bounded below by the minimum honest quality and the weighted median is within the honest range, the result follows. $\square$

5.5 Attack Analysis

Random Noise Attack. Byzantine nodes submit random $\Delta_j^{(s)}(v) \sim \text{Uniform}(\{-1, +1\})$. Since honest nodes have correlated submissions (reflecting genuine patterns), the weighted median filters out uncorrelated noise. The expected degradation is $O(f/n)$ in prior quality.

Targeted Poisoning. Byzantine nodes identify high-value schema elements and submit adversarial factors designed to invert the correct recommendation. By Theorem 2, this requires controlling $> W_H/2$ total weight, which is impossible under the stake and quality constraints.

Gradient Mimicry. Sophisticated adversaries might produce plausible-looking priors that subtly degrade performance. Quality verification via holdout evaluation (Section 4, Phase 3) detects such attacks with probability $\geq 1 - (1 - \delta)^k$ after k validation samples.

6 Sybil Resistance

6.1 Stake-Based Identity

Each participant must stake $D_{\min}$ tokens (default: 25 \$AI) per submission. Sybil attacks require splitting stake across multiple identities.

Proposition 4 (Sybil Cost). *An adversary with total stake S_{adv} creating k Sybil identities has per-identity stake S_{adv}/k . Since weights are proportional to stake (Eq. 6), the total adversarial weight is:*

$$W_{adv} = \sum_{j=1}^k \frac{S_{adv}}{k} \cdot q_j = S_{adv} \cdot \bar{q}_{adv}, \quad (10)$$

which is independent of k . Sybil splitting provides no advantage in weighted median aggregation.

6.2 Quality Gate

Submissions with quality $q_i < q_{\min}$ (default: 0.3) are filtered before aggregation. This creates a two-tier defense:

1. **Bond requirement:** Each submission requires stake, making mass submission expensive.
2. **Quality threshold:** Low-quality submissions are discarded regardless of stake, and their bonds are subject to slashing.

Theorem 5 (Sybil Resistance). *Under the DSO protocol, an adversary needs total stake*

$$S_{adv} > \frac{W_H}{2\bar{q}_{adv}} \quad (11)$$

to influence the aggregated prior, where W_H is total honest weight and $\bar{q}_{adv}$ is the average quality score of adversarial submissions. If the adversary's submissions are random noise, $\bar{q}_{adv} \approx 0.5$, requiring $S_{adv} > W_H$. If quality gating rejects adversarial submissions (i.e., $\bar{q}_{adv} < q_{\min}$), the adversary has zero influence regardless of stake.

6.3 Bond and Slashing Mechanism

Submission of prior E_i requires bond $D_i \geq D_{\min}$. The bond is governed by:

Definition 6 (Slashing Conditions). *Bond D_i is slashed (partially burned) when:*

1. *The validated quality $q'_i < q_{\min}$ and a challenger submits a valid proof (counter-examples on holdout tasks).*
2. *The Merkle proof verification fails (data integrity violation).*
3. *The submission is identified as duplicate of a previous entry.*

The slashing procedure:

$$\text{Burn amount} = \sigma \cdot D_i \quad (\text{default } \sigma = 0.5), \quad (12)$$

$$\text{Challenger reward} = (1 - \sigma) \cdot D_i, \quad (13)$$

$$\text{Remaining} = 0. \quad (14)$$

This creates a bounty for identifying malicious submissions, incentivizing active monitoring by honest participants.

7 Gossip Protocol

7.1 Protocol Design

DSO uses an epidemic gossip protocol [7] for propagating high-quality priors across the network. The protocol is designed to be:

- **Pull-based:** Nodes request priors they lack, avoiding unsolicited data transfer.
- **Quality-filtered:** Only priors with $q_i \geq q_{\text{fetch}}$ (default: 0.5) are propagated.
- **Stake-weighted:** Propagation priority is proportional to quality score times stake.

Algorithm 3 DSO Gossip Protocol (per node)

Require: Local replica L , registry connection, peer set $\mathcal{P}$

```
1: Subscribe to registry events: NewSubmission, QualityUpdate
2: loop
3:   events  $\leftarrow$  Registry.PollEvents()
4:   for each NewSubmission(id, q, cid) in events do
5:     if  $q \geq q_{\text{fetch}}$  and  $\text{id} \notin L$  then
6:        $E \leftarrow$  FetchFromPeersOrIPFS(cid,  $\mathcal{P}$ )
7:       Verify:  $\text{MerkleRoot}(E) = \text{Registry.GetMerkleRoot}(\text{id})$ 
8:       if verification passes then
9:          $L.\text{Insert}(E, \text{id})$ 
10:        Announce availability to peers
11:      end if
12:    end if
13:  end for
14:  for each QualityUpdate(id,  $q_{\text{new}}$ ) in events do
15:    if  $q_{\text{new}} < q_{\text{min}}$  and  $\text{id} \in L$  then
16:       $L.\text{Remove}(\text{id})$  ▷ Garbage collect low-quality
17:    end if
18:  end for
19:  Sleep( $\Delta t_{\text{gossip}}$ ) ▷ Default: 30s
20: end loop
```

7.2 Propagation Analysis

Proposition 6 (Propagation Time). *In a network of n nodes with random peer connections (average degree d), a new prior with quality $q \geq q_{\text{fetch}}$ reaches all honest nodes in expected time:*

$$T_{\text{prop}} = O\left(\frac{\log n}{d} \cdot \Delta t_{\text{gossip}}\right). \quad (15)$$

Proof. Standard epidemic gossip analysis [7]. Each gossip round, each informed node tells d peers. The number of informed nodes doubles each round until saturation, giving $O(\log n)$ rounds. With gossip interval Δt_{gossip} and average degree d , the effective round time is $\Delta t_{\text{gossip}}/d$. $\square$

For typical parameters ($n = 1000$, $d = 10$, $\Delta t = 30\text{s}$), propagation time is approximately $O(\log(1000)/10 \times 30) \approx 21\text{s}$.

7.3 Bandwidth Efficiency

The 1-bit compression from ASO is critical for gossip efficiency:

7.4 Incentive-Compatible Propagation

Nodes that propagate high-quality priors earn fee rebates:

$$\text{rebate}_i = \gamma_{\text{prop}} \sum_{j: \text{fetched from } i} q_j \cdot \text{size}_j, \quad (16)$$

where γ_{prop} is the propagation reward rate (default: 0.001 \$AI per quality-byte). This incentivizes nodes to maintain high availability and serve popular priors.

Representation	Size per Prior	Bandwidth (100 priors)
Full precision (FP32)	128 KB	12.5 MB
16-bit quantized	64 KB	6.25 MB
1-bit compressed	4.3 KB	430 KB
1-bit + schema sparse	1.2 KB	120 KB

Table 1: Prior data sizes under different representations. 1-bit compression with schema-sparse encoding reduces bandwidth by 100× compared to full precision.

8 Convergence Analysis

8.1 Aggregation Convergence

We prove that the aggregated prior converges to the honest-majority consensus as the number of participants grows.

Theorem 7 (Aggregation Convergence). *Let Δ^* be the true optimal prior for the task distribution. Under Assumption 1, with n total participants ($n - f$ honest), the aggregated prior satisfies:*

$$\mathbb{E} \left[\|\tilde{\Delta} - \Delta^*\|_1 \right] \leq \frac{C}{\sqrt{n-f}} + O\left(\frac{f}{n}\right), \quad (17)$$

where C depends on the variance of honest submissions.

Proof. Decompose the error into honest estimation error and Byzantine corruption:

$$\|\tilde{\Delta} - \Delta^*\|_1 \leq \underbrace{\|\tilde{\Delta} - \Delta_H^*\|_1}_{\text{Byzantine deviation}} + \underbrace{\|\Delta_H^* - \Delta^*\|_1}_{\text{honest estimation error}}, \quad (18)$$

where Δ_H^* is the weighted median of honest submissions only.

Byzantine deviation: By Theorem 2, the weighted median with Byzantine nodes equals the weighted median of a subset of honest values. The deviation from the full honest median is bounded by the gap between adjacent honest submissions, which is $O(1/\sqrt{n-f})$ by concentration.

Honest estimation error: The honest median converges to Δ^* at rate $O(1/\sqrt{n-f})$ by standard median convergence for bounded random variables [18].

The $O(f/n)$ term accounts for the influence of the quality gate: some honest priors near the quality boundary may be incorrectly filtered when Byzantine nodes manipulate quality votes. $\square$

8.2 Prior Quality Dynamics

The quality of the aggregated prior bank evolves over time as new submissions arrive and old ones are pruned.

Proposition 8 (Quality Monotonicity). *Under the condition that the expected quality of new submissions exceeds the current aggregate quality:*

$$\mathbb{E}[q_{\text{new}}] > \tilde{q}_{\text{current}}, \quad (19)$$

the aggregate quality $\tilde{q}$ is non-decreasing in expectation over successive aggregation rounds.

8.3 Network Effect

As more agents join the network and contribute priors, each individual agent benefits from the collective knowledge:

Corollary 9 (Network Scaling). *The expected performance improvement for each agent scales as:*

$$\Delta_{\text{perf}}(n) = \Theta \left(\sqrt{\frac{n-f}{n_0}} \right) \cdot \Delta_{\text{perf}}(n_0), \quad (20)$$

where n_0 is the baseline number of agents and $\Delta_{\text{perf}}(n_0)$ is the improvement with n_0 agents. This sub-linear scaling reflects diminishing returns from additional priors as the prior bank becomes saturated.

9 Incentive Mechanisms and PoAI Integration

9.1 Proof-of-AI (PoAI) Attestations

Each prior submission includes a PoAI attestation containing:

- **TEE report:** Trusted execution environment attestation proving the prior was generated by legitimate ASO execution.
- **Task metrics:** Information gain ΔI , utility improvement ΔU , and compute cost.
- **Holdout results:** Performance on a reserved validation set.

9.2 Emission Formula

The reward for submitting prior i is:

$$R_i = \gamma \Delta I_i + \beta \Delta U_i - \lambda_c \cdot \text{cost}_i, \quad (21)$$

where:

- $\gamma = 1.0$: Weight for information gain (measured as KL divergence between prior and posterior task distributions).
- $\beta = 0.5$: Weight for utility improvement (measured as pass-rate improvement on holdout tasks).
- $\lambda_c = 0.1$: Cost penalty (discouraging wasteful compute).

9.3 Quality Score Evolution

Quality scores evolve via exponential moving average:

$$q_i^{(t+1)} = (1 - \rho) \cdot q_i^{(t)} + \rho \cdot q_i^{\text{new}}, \quad (22)$$

where $\rho = 0.1$ is the update rate and q_i^{new} is the latest validation score. This smooths out noise while allowing quality to track genuine changes.

9.4 Game-Theoretic Analysis

Proposition 10 (Honest Submission is Nash Equilibrium). *Under the DSO incentive structure, honest prior submission is a Nash equilibrium when the expected reward for honest submission exceeds the expected reward for any deviating strategy:*

$$\mathbb{E}[R_{\text{honest}}] - D_{\min} > \mathbb{E}[R_{\text{deviate}}] - \sigma D_{\min}. \quad (23)$$

This holds when $\gamma\Delta I_{\text{honest}} + \beta\Delta U_{\text{honest}} > (1 - \sigma)D_{\min}$, i.e., when honest priors generate meaningful information gain and utility improvement relative to the bond.

10 Experimental Evaluation

10.1 Setup

We evaluate DSO on multi-agent code generation using SWE-bench Verified [11] (500 issues). The setup:

- **Agents:** 10 Claude 3.5 Sonnet instances running ASO with group size $G = 4$ and $K = 3$ iterations.
- **Task distribution:** Issues distributed round-robin across agents.
- **Network:** Simulated P2P network with 50ms latency and 100Mbps bandwidth per link.
- **Byzantine configurations:** 0%, 10%, 20%, 30% of agents are Byzantine (submitting random priors).

10.2 Multi-Agent Code Generation

Configuration	Resolved Rate	Avg. Prior Reuse	Δ vs. Isolated
Isolated agents (no DSO)	16.3%	0.0	–
DSO, 0% Byzantine	18.8%	3.2 priors/task	+2.5%
DSO, 10% Byzantine	18.4%	3.0 priors/task	+2.1%
DSO, 20% Byzantine	17.9%	2.8 priors/task	+1.6%
DSO, 30% Byzantine	17.1%	2.4 priors/task	+0.8%
DSO + median voting, 0% Byz.	18.7%	3.1 priors/task	+2.4%
DSO + median voting, 20% Byz.	18.5%	2.9 priors/task	+2.2%
DSO + mean voting, 20% Byz.	16.8%	2.6 priors/task	+0.5%

Table 2: SWE-bench Verified results (500 issues), 10 agents. Median voting maintains performance under 20% Byzantine faults, while mean voting degrades significantly. The 15.2% improvement headline figure compares DSO (0% Byz.) aggregate performance to individual isolated agents.

Aggregation	0% Byz.	20% Byz.	30% Byz.
No aggregation (isolated)	16.3%	16.3%	16.3%
Simple mean	18.6%	16.8%	15.1%
Trimmed mean (10%)	18.5%	17.5%	16.4%
Coordinate-wise median	18.4%	18.1%	17.0%
Weighted median (DSO)	18.7%	18.5%	17.1%
Krum [4]	18.2%	17.8%	16.9%

Table 3: Comparison of aggregation methods under varying Byzantine fractions. Weighted median (DSO) achieves the best robustness-performance trade-off.

Component	Per Agent	Total (10 agents)
Full-precision priors	2.4 GB	24 GB
1-bit compressed	82 MB	820 MB
IPFS overhead (+metadata, proofs)	+10 MB	+100 MB
Local replica (after dedup)	45 MB	450 MB
Effective compression	29.3$\times$ (raw), 26$\times$ (with overhead)	

Table 4: Storage costs per agent and total across the 10-agent network.

10.3 Aggregation Method Comparison

10.4 Storage Efficiency

10.5 Propagation Latency

10.6 Scaling with Network Size

11 Related Work

Federated Learning. FedAvg [16] aggregates local model updates via averaging. FedProx [15] adds a proximal term for heterogeneous settings. These methods aggregate gradients/parameters, unlike DSO which aggregates semantic priors.

Byzantine-Robust Aggregation. Krum [4] selects the update closest to others. Bulyan [17] combines Krum with trimmed mean. [22] analyze coordinate-wise median and trimmed mean. Our weighted median extends these ideas to semantic prior aggregation.

Decentralized Machine Learning. Swarm Learning [20] coordinates peer-to-peer training without a central server. Gossip-based SGD [12] distributes gradient computation. DSO differs by operating on compressed semantic priors rather than model parameters.

Blockchain for ML. FLock [8] uses blockchain for federated learning coordination. Bittensor [3] creates a decentralized marketplace for ML models. DSO uses blockchain only for integrity and incentive management, keeping compute and storage off-chain.

Network Size	Avg. Latency	P95 Latency	Bandwidth
10 nodes	2.3s	4.1s	48 KB/round
50 nodes	5.8s	11.2s	240 KB/round
100 nodes	8.4s	16.7s	480 KB/round
1000 nodes	14.2s	28.3s	4.8 MB/round

Table 5: Gossip propagation latency for a single prior (1.2 KB compressed). Latency scales logarithmically with network size as predicted by Proposition 6.

Agents	Resolved Rate	Improvement	Priors/Task
1 (isolated ASO)	16.3%	–	0.0
5	17.8%	+1.5%	1.8
10	18.7%	+2.4%	3.1
25	19.4%	+3.1%	5.2
50	19.8%	+3.5%	6.8
100	20.1%	+3.8%	7.5

Table 6: Performance scaling with network size (0% Byzantine). Diminishing returns are consistent with the $O(\sqrt{n})$ scaling predicted by Corollary 7.

Parameter-Efficient Federated Learning. FedLoRA [23] and FedPara [10] federate LoRA or low-rank parameter updates. These still require gradient computation; DSO is entirely gradient-free.

Experience Sharing in Multi-Agent RL. [5] and [1] study experience sharing in multi-agent reinforcement learning. DSO adapts these ideas to the LLM setting with compressed semantic priors instead of raw trajectories.

12 Conclusion

Decentralized Semantic Optimization enables collective model adaptation by sharing compressed experiential priors across a network of LLM agents. The three-layer storage architecture balances integrity, availability, and performance. Byzantine-robust weighted median aggregation provides formal guarantees under adversarial conditions. Sybil resistance via stake-weighted quality gating prevents identity-based attacks.

Key Results.

- 15.2% improvement in multi-agent code generation vs. isolated agents.
- Resilience to 20% Byzantine participants (degradation < 0.9%).
- $29.3\times$ storage compression via 1-bit quantization.
- Logarithmic propagation latency scaling with network size.

Limitations. DSO requires at least $2f + 1$ honest participants for guaranteed correctness. The quality verification relies on holdout benchmarks, which may not represent all task types. Cross-domain transfer (e.g., Python $\rightarrow$ Rust) remains limited.

Future Work. Cross-domain prior transfer via embedding alignment, hierarchical aggregation with specialized sub-groups, privacy-preserving aggregation via secure multi-party computation, and integration with Hamiltonian Market Maker (HMM) for pricing prior contributions.

A Security Analysis (Extended)

A.1 Formal Sybil Resistance Proof

Theorem 11 (Sybil Resistance under Stake Splitting). *Let adversary $\mathcal{A}$ have total stake S_A and create k identities with stakes $S_1, \dots, S_k$ where $\sum_j S_j = S_A$. For any aggregation function F that is a weighted median with weights proportional to stake, the output $F(\{E_i\}_{i \in H} \cup \{E'_j\}_{j=1}^k)$ is identical regardless of k .*

Proof. The weighted median depends only on the total weight on each side of any candidate value. For adversarial values, the total weight is:

$$W_A = \sum_{j=1}^k S_j \cdot q_j \leq S_A \cdot \max_j q_j, \quad (24)$$

which is maximized when all stake is concentrated ($k = 1$) with the highest possible quality score. Splitting stake across $k > 1$ identities can only reduce total weight (if quality varies) or keep it the same (if all identities achieve the same quality). Thus, Sybil splitting is weakly dominated. $\square$

A.2 Data Poisoning Resistance

Proposition 12 (Poisoning Detection). *A targeted poisoning attack that inverts p fraction of a prior’s recommendations is detected by holdout validation with probability:*

$$P(\text{detect}) = 1 - (1 - p)^k \geq 1 - e^{-pk}, \quad (25)$$

where k is the number of holdout validation samples. For $p = 0.1$ and $k = 30$, detection probability is $\geq 95.2\%$.

A.3 Denial of Service Resistance

The bond requirement creates a linear cost for submission flooding:

$$\text{Cost}_{\text{flood}}(m) = m \cdot D_{\min} + \text{gas}(m), \quad (26)$$

where m is the number of spam submissions. At $D_{\min} = 25$ \$AI and typical gas costs, flooding 10,000 submissions costs $> 250,000$ \$AI, making sustained DoS economically infeasible.

B Gossip Protocol Formal Analysis

Theorem 13 (Gossip Convergence). *In a connected graph with n nodes and minimum degree $d_{\min} \geq 3$, the pull-based gossip protocol (Algorithm 3) ensures that a new prior reaches all honest nodes within:*

$$T \leq \left\lceil \frac{3 \ln n}{\ln d_{\min}} \right\rceil \cdot \Delta t_{\text{gossip}} \quad (27)$$

rounds with probability $\geq 1 - 1/n$.

Proof. At each round, an uninformed node contacts $d_{\min}$ random peers. The probability that at least one peer is informed is $1 - (1 - p_t)^{d_{\min}}$ where p_t is the fraction of informed nodes at time t . Using the standard rumor spreading analysis [7], the fraction of informed nodes satisfies:

$$p_{t+1} \geq 1 - (1 - p_t) \cdot (1 - p_t)^{d_{\min}} = 1 - (1 - p_t)^{d_{\min}+1}. \quad (28)$$

Starting from $p_0 = 1/n$, after $O(\log n / \log d_{\min})$ rounds, $p_t > 1 - 1/n$. $\square$

C Full Contract Specification

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface IExperienceRegistry {
    struct Entry {
        bytes32 merkleRoot;
        string cid;
        uint64 schema;
        uint64 quality; // [0, 1000] fixed-point
        address submitter;
        uint256 bond;
        uint64 timestamp;
        uint8 status; // 0=pending, 1=verified, 2=slashed
    }

    event Submitted(uint256 indexed id, address indexed submitter);
    event QualityVoted(uint256 indexed id, address voter, uint64 score);
    event Challenged(uint256 indexed id, address challenger, bool slashed);
    event Aggregated(bytes32 aggregateRoot, uint256[] sourceIds);

    function submit(Entry calldata e) external payable
        returns (uint256 id);

    function voteQuality(uint256 id, uint64 score) external;

    function challenge(uint256 id, bytes calldata proof)
        external returns (bool slashed);

    function aggregate(uint256[] calldata ids)
```

```

        external returns (bytes32 aggregateMerkleRoot);

function getEntry(uint256 id)
    external view returns (Entry memory);

function getEntriesBySchema(uint64 schema)
    external view returns (uint256[] memory ids);

function getSubmitterEntries(address submitter)
    external view returns (uint256[] memory ids);

function getAggregateRoot(uint64 schema)
    external view returns (bytes32);
}

```

D Reproducibility Details

- **Environment:** Docker containers with Python 3.11, Go 1.21, LanceDB 0.3.0, IPFS Kubo 0.25.0.
- **Network simulation:** Custom simulator with configurable latency, bandwidth, and packet loss.
- **Byzantine behavior:** Random noise ($\Delta \sim \text{Unif}(\{-1, +1\})$) for all reported experiments. Targeted attacks evaluated in Appendix A.
- **Seeds:** Fixed seeds (42, 123, 456) for three runs; results report mean $\pm$ std (std $< 0.3\%$ in all cases).
- **Hardware:** 10 $\times$ M1 MacBook Pro (simulated agents), 1 $\times$ 4-core VM for registry and IPFS.

References

- [1] Baker, B., Kanitscheider, I., Marber, T., et al. Emergent tool use from multi-agent autocurricula. *ICLR*, 2020.
- [2] Benet, J. IPFS — Content addressed, versioned, P2P file system. *arXiv preprint arXiv:1407.3561*, 2014.
- [3] Bittensor. Bittensor: A peer-to-peer intelligence market. Whitepaper, 2023.
- [4] Blanchard, P., El Mhamdi, E. M., Guerraoui, R., and Stainer, J. Machine learning with adversaries: Byzantine tolerant gradient descent. *NeurIPS*, 2017.
- [5] Christianos, F., Papoudakis, G., Rahman, M. A., and Albrecht, S. V. Scaling multi-agent reinforcement learning with selective parameter sharing. *ICML*, 2021.
- [6] David, H. A. and Nagaraja, H. N. *Order Statistics*. Wiley, 3rd edition, 2003.
- [7] Demers, A., Greene, D., Hauser, C., et al. Epidemic algorithms for replicated database maintenance. *PODC*, 1987.

- [8] FLock. FLock: Federated learning on blockchain. <https://flock.io>, 2024.
- [9] Hanzo AI Research. Active Semantic Optimization: Training-free adaptation via Bayesian Product-of-Experts decoding. Technical report, Hanzo AI, 2026.
- [10] Hyeon-Woo, N., Ye-Bin, M., and Tae-Hyun, O. FedPara: Low-rank Hadamard product for communication-efficient federated learning. *ICLR*, 2022.
- [11] Jiménez, C. E., Yang, J., Wettig, A., et al. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.
- [12] Koloskova, A., Stich, S. U., and Jaggi, M. Decentralized stochastic optimization and gossip algorithms with compressed communication. *ICML*, 2020.
- [13] Lamport, L., Shostak, R., and Pease, M. The Byzantine generals problem. *ACM TOPLAS*, 4(3):382–401, 1982.
- [14] LanceDB. LanceDB: Developer-friendly serverless vector database. <https://lancedb.github.io/lancedb/>, 2024.
- [15] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., and Smith, V. Federated optimization in heterogeneous networks. *MLSys*, 2020.
- [16] McMahan, B., Moore, E., Ramage, D., Hampson, S., and Arcas, B. A. y. Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 2017.
- [17] El Mhamdi, E. M., Guerraoui, R., and Rouault, S. The hidden vulnerability of distributed learning in Byzantium. *ICML*, 2018.
- [18] Minsker, S. Geometric median and robust estimation in Banach spaces. *Bernoulli*, 21(4):2308–2335, 2015.
- [19] Pease, M., Shostak, R., and Lamport, L. Reaching agreement in the presence of faults. *JACM*, 27(2):228–234, 1980.
- [20] Warnat-Herresthal, S., Schultze, H., Shastri, K. L., et al. Swarm Learning for decentralized and confidential clinical machine learning. *Nature*, 594:265–270, 2021.
- [21] Williams, S. Arweave: A protocol for economically sustainable information permanence. Yellowpaper, 2019.
- [22] Yin, D., Chen, Y., Kannan, R., and Bartlett, P. Byzantine-robust distributed learning: Towards optimal statistical rates. *ICML*, 2018.
- [23] Zhang, Y., Chen, X., Wang, Y., et al. FedLoRA: Communication-efficient federated learning with low-rank adaptation. *arXiv preprint arXiv:2402.11846*, 2024.